



Department of Computer Science
COMP438 — Distributed Computing

Peer-to-Peer Chat with a Directory Server

Course Project — Phase 1

Baraa Rjoob 1221076
Abdallah Alqasrawi 1221432

Instructor: Dr. Mohammad Helal

1 Design

The system has two programs. The **Directory** is a rendezvous server: it tracks which clients are alive and brokers their network credentials, but it never carries chat traffic. Each **Client** keeps one control connection to the Directory and opens *direct* TCP connections to other clients for chat.

On startup a client (i) connects to the Directory, (ii) opens its own listening socket on an OS-assigned port for incoming peers, and (iii) sends an **ALIVE** message every second carrying its id and that peer port. The Directory records each client's IP, peer port, and last-seen time, and pushes the list of active client ids to every client once per second. To chat, a client asks the Directory for a target's IP and port, then connects to it directly.

Rationale. Messages are newline-delimited JSON because it is self-describing and trivial to frame; a thread-per-connection model keeps the code simple and lets the UI thread run without blocking on I/O; routing chat *directly* between peers keeps the Directory lightweight and removes it as a traffic bottleneck. Usage: `python3 directory.py [port]` and `python3 client.py <id> [host] [port]`.

2 Sockets and Concurrency

Socket count.

- *Directory*: 1 listening socket, plus 1 control socket per connected client.
- *Client*: 1 control socket to the Directory, 1 listening socket for incoming peers, and 1 socket per active peer chat.

Concurrency. Both programs use a thread-per-connection model.

- The Directory accepts each control connection in its own handler thread, plus two background threads: a broadcaster (sends the active list) and a reaper (drops clients that stop sending **ALIVE**, i.e. soft state).
- Each client runs concurrent threads for the heartbeat, the Directory listener, the peer acceptor, and one reader per connected peer, so the input loop never blocks on the network.
- Shared registries (the Directory's presence/connection tables, the client's peer table) are guarded by a mutex. Writes to each socket are serialized by a per-socket lock so concurrent threads cannot interleave bytes from two frames on the same connection.
- On disconnect, the reader thread removes the peer and clears chat focus; the Directory's reaper removes any client silent beyond the timeout.

Timing parameters.

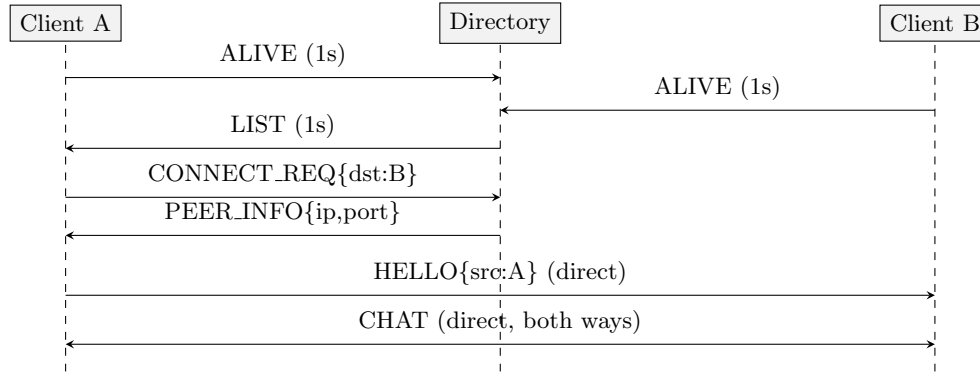
Parameter	Value	Meaning
Heartbeat interval	1 s	client → Directory ALIVE period
List broadcast	1 s	Directory → clients active-list period
Alive timeout	3 s	silence after which a client is reaped

A client therefore disappears from every active list within roughly 3–4 s of going silent.

3 Establishing a Client-to-Client Connection

1. Client A issues `/chat B`, sending `CONNECT_REQ{dst:B}` to the Directory.
2. The Directory looks up B's record and replies with `PEER_INFO{ip, port}` (or `ERROR` if B is unknown/inactive).

3. A opens a direct TCP connection to B's advertised IP:port and sends `HELLO{src:A}` as its first frame, so B learns who connected.
 4. Both sides then exchange `CHAT` messages directly, with no further Directory involvement.
- If both clients request each other at the same time, two connections may form; the first one registered is kept and the redundant socket is closed, so a pair of clients shares exactly one direct connection.



4 Application-Layer Header

Every message is one JSON object terminated by a newline (`\n`). Newline framing is unambiguous because JSON encoding escapes any newline inside a string, so a literal `\n` never appears within a frame. The mandatory `type` field acts as the header that selects how the remaining fields are interpreted.

type	Direction	Fields	Purpose
ALIVE	client → dir	src, peer_port	heartbeat + register
LIST	dir → client	clients[]	active ids (names only)
CONNECT_REQ	client → dir	src, dst	request a peer's address
PEER_INFO	dir → client	dst, ip, port	peer's IP and port
ERROR	dir → client	reason	target unknown/inactive
HELLO	client → peer	src	identify on a new peer link
CHAT	client → peer	src, text	chat message

5 Sample Run

Three processes on one host: one Directory and two clients, `alice` and `bob`. Both register and heartbeat; `alice` lists the active clients, opens a direct chat to `bob`, sends a message, then quits.

Directory

```

[directory] listening on 0.0.0.0:5071
[directory] registered 'bob' at 127.0.0.1:41437
[directory] registered 'alice' at 127.0.0.1:34223
[directory] alice -> connect info for bob
[directory] 'bob' disconnected
[directory] 'alice' disconnected
  
```

Client alice

```
[client] 'alice' control->127.0.0.1:5071 peer-listen port=34223
Commands: /list /chat <id> /quit
> Active clients: alice, bob
> /chat bob
[connected] now chatting with 'bob'
> hello bob
> /quit
[disconnected] 'bob' left the chat
[client] bye
```

Client bob

```
[client] 'bob' control->127.0.0.1:5071 peer-listen port=41437
Commands: /list /chat <id> /quit
[connected] 'alice' connected to you
[alice] hello bob
[client] bye
```